Table of Contents

Project Description 2
Dataset Description 2
Summary of Data Analysis 3
MapReduce Analysis 3
Pig Analysis 13
Conclusion 20

Project Description

This project focuses on the Big Data analysis of NYC 311 Service Requests using the Hadoop ecosystem. Approximately 3.85 million service request records were stored and processed using HDFS for distributed data storage. Apache MapReduce was used to perform large scale aggregations such as analyzing requests by agency and submission channel, while Apache Pig was used to perform various data analysis tasks including identifying common complaint types, borough wise requests, resolution times, monthly trends, complaint statuses and complaint locations. The project shows how Hadoop based technologies can efficiently store, process and analyze large real world datasets to identify useful patterns and insights.

Dataset Description

Dataset: https://data.cityofnewyork.us/Social-Services/311-Service-Requests-from-2020-to-Present/erm2-nwe9/about_data

Dataset Description:

Item	Description
Dataset Name	311 Service Requests from 2020 to Present
Source	NYC Open Data – City of New York
Dataset ID	erm2-nwe9
Data Type	311 Service Requests
Original Dataset Size	22.3+ million rows
Current Dataset Columns	44
Number of Records Used	3,850,000 NYC 311 service request records distributed across 77 CSV files
Purpose	Contains service requests reported through NYC 311 and the agencies responsible for handling them.

Key columns:

Column Name	Description
Unique Key	Unique identifier assigned to each service request.
Created Date	Date and time when the service request was created.
Closed Date	Date and time when the service request was closed.
Agency	Code of the city agency responsible for the request.
Agency Name	Full name of the responsible city agency.
Complaint Type	Main category or type of the reported problem.
Descriptor	Provides additional details about the complaint.
Location Type	Type of location where the incident occurred.
Incident Zip	ZIP code associated with the service request.

Borough	NYC borough where the incident occurred.
Status	Current status of the service request.
Resolution Description	Description of how the service request was resolved.
Channel	Method used to submit the service request, such as phone, online, or mobile.
Latitude	Latitude coordinate of the incident location.
Longitude	Longitude coordinate of the incident location.

Summary of Data Analysis

The following analyses were performed using Hadoop MapReduce:

1. 311 Service Requests by Borough
2. 311 Service Requests by Agency
3. 311 Service Requests by Submission Channel

The following analyses were performed using Apache Pig:

1. Top Complaint Types
2. Top Complaint Locations
3. Average Complaint Resolution Time
4. Monthly Complaint Trends
5. 311 Service Requests by Borough
6. Complaint Status Distribution

MapReduce Analysis

1. **Complaints by Borough:** Counts the number of service requests reported in each NYC borough to compare complaint volumes geographically.

   Code:

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class BoroughCount {
```

```
// Mapper
public static class BoroughMapper
      extends Mapper<Object, Text, Text, IntWritable> {

   private final static IntWritable one = new IntWritable(1);
   private Text borough = new Text();

   public void map(Object key, Text value, Context context)
         throws IOException, InterruptedException {

      String line = value.toString();

      // Skip CSV header
      if (line.startsWith("\"unique_key\"")) {
         return;
      }

      // Split CSV line while keeping quoted fields together
      String[] fields = line.split(",(?=(?:[^\"]*\"[^\"]*\")*[^\"]*$)");

      // Borough is column 29 (index 28)
      if (fields.length > 28) {

         String boroughValue = fields[28]
               .replace("\"", "")
               .trim();

         if (!boroughValue.isEmpty()) {
            borough.set(boroughValue);
            context.write(borough, one);
         }
      }
   }
}

// Reducer
public static class BoroughReducer
      extends Reducer<Text, IntWritable, Text, IntWritable> {

   private IntWritable result = new IntWritable();

   public void reduce(Text key, Iterable<IntWritable> values,
                      Context context)
         throws IOException, InterruptedException {

      int sum = 0;

      for (IntWritable value : values) {
```

```
            sum += value.get();
        }

        result.set(sum);
        context.write(key, result);
    }
}

// Driver
public static void main(String[] args)
        throws Exception {

    if (args.length != 2) {
        System.err.println(
                "Usage: BoroughCount <input path> <output path>");
        System.exit(2);
    }

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "NYC 311 Complaints by Borough");

    job.setJarByClass(BoroughCount.class);

    job.setMapperClass(BoroughMapper.class);
    job.setReducerClass(BoroughReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}
```

Execution:

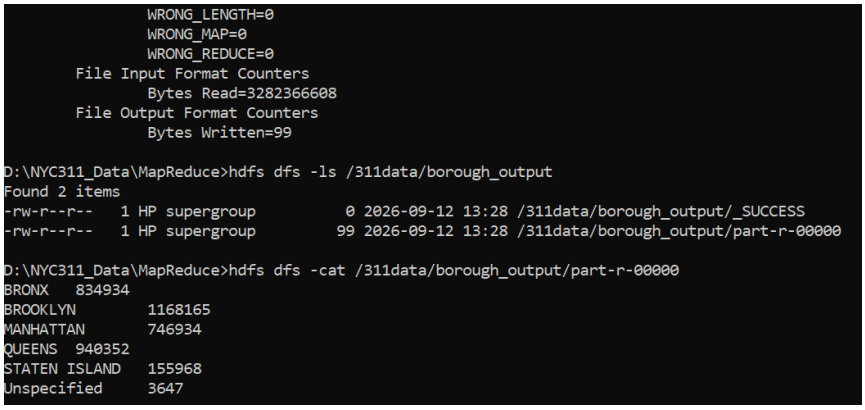


Figure: Output of borough_output

2. **Requests by Agency:** Counts requests handled by each city agency to identify which agencies receive the highest number of service requests.

Code:

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class AgencyCount {

   public static class AgencyMapper
         extends Mapper<Object, Text, Text, IntWritable> {

      private final static IntWritable one = new IntWritable(1);
      private Text agency = new Text();

      public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

         String line = value.toString();

         // Skip CSV header
         if (line.startsWith("\"unique_key\"")) {
            return;
         }

         // Handle commas inside quoted CSV fields
         String[] fields =
               line.split(",(?=(?:[^\"]*\"[^\"]*\")*[^\"]*$)");

         // Agency is column 4 (index 3)
         if (fields.length > 3) {

            String agencyValue = fields[3]
                  .replace("\"", "")
                  .trim();

            if (!agencyValue.isEmpty()) {
               agency.set(agencyValue);
```

```
            context.write(agency, one);
          }
        }
    }
}

public static class AgencyReducer
      extends Reducer<Text, IntWritable, Text, IntWritable> {

    private IntWritable result = new IntWritable();

    public void reduce(Text key,
                       Iterable<IntWritable> values,
                       Context context)
          throws IOException, InterruptedException {

        int sum = 0;

        for (IntWritable value : values) {
          sum += value.get();
        }

        result.set(sum);
        context.write(key, result);
    }
}

public static void main(String[] args)
      throws Exception {

    if (args.length != 2) {
        System.err.println(
              "Usage: AgencyCount <input path> <output path>");
        System.exit(2);
    }

    Configuration conf = new Configuration();

    Job job = Job.getInstance(
          conf,
          "NYC 311 Requests by Agency");

    job.setJarByClass(AgencyCount.class);

    job.setMapperClass(AgencyMapper.class);
    job.setReducerClass(AgencyReducer.class);

    job.setOutputKeyClass(Text.class);
```

```
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(
                job,
                new Path(args[0]));

        FileOutputFormat.setOutputPath(
                job,
                new Path(args[1]));

        System.exit(
                job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Execution:

```
D:\NYC311_Data>cd /d D:\NYC311_Data\MapReduce

D:\NYC311_Data\MapReduce>notepad AgencyCount.java

D:\NYC311_Data\MapReduce>javac -cp "%HADOOP_HOME%\share\hadoop\common\*;%HADOOP_HOME%\share\hadoop\common\lib\*;%HADOOP_
HOME%\share\hadoop\hdfs\*;%HADOOP_HOME%\share\hadoop\mapreduce\*;%HADOOP_HOME%\share\hadoop\yarn\*" AgencyCount.java

D:\NYC311_Data\MapReduce>jar -cvf AgencyCount.jar AgencyCount*.class
added manifest
adding: AgencyCount$AgencyMapper.class(in = 2063) (out= 918)(deflated 55%)
adding: AgencyCount$AgencyReducer.class(in = 1745) (out= 738)(deflated 57%)
adding: AgencyCount.class(in = 1645) (out= 913)(deflated 44%)

D:\NYC311_Data\MapReduce>dir AgencyCount.jar
 Volume in drive D is New Volume
 Volume Serial Number is A2F4-ACAC

 Directory of D:\NYC311_Data\MapReduce

09/12/2026  10:45 PM             3,343 AgencyCount.jar
               1 File(s)          3,343 bytes
               0 Dir(s)  140,935,573,504 bytes free

D:\NYC311_Data\MapReduce>
```

```
D:\NYC311_Data\MapReduce>hadoop jar AgencyCount.jar AgencyCount /311data/input /311data/agency_output
```

Figure: Commands for creating java file, jar file and running jar file

```
2026-09-12 23:00:54,239 INFO mapreduce.Job:  map 93% reduce 31%
2026-09-12 23:00:57,407 INFO mapreduce.Job:  map 94% reduce 31%
2026-09-12 23:01:07,654 INFO mapreduce.Job:  map 95% reduce 31%
2026-09-12 23:01:14,890 INFO mapreduce.Job:  map 96% reduce 31%
2026-09-12 23:01:18,985 INFO mapreduce.Job:  map 97% reduce 32%
2026-09-12 23:01:23,123 INFO mapreduce.Job:  map 98% reduce 32%
2026-09-12 23:01:27,189 INFO mapreduce.Job:  map 99% reduce 32%
2026-09-12 23:01:31,248 INFO mapreduce.Job:  map 99% reduce 33%
2026-09-12 23:01:34,288 INFO mapreduce.Job:  map 100% reduce 33%
2026-09-12 23:01:37,324 INFO mapreduce.Job:  map 100% reduce 100%
2026-09-12 23:01:37,329 INFO mapreduce.Job: Job job_1789197051950_0002 completed successfully
2026-09-12 23:01:37,435 INFO mapreduce.Job: Counters: 55
        File System Counters
                FILE: Number of bytes read=40721888
                FILE: Number of bytes written=100059958
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=3282375694
                HDFS: Number of bytes written=174
                HDFS: Number of read operations=236
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
                HDFS: Number of bytes read erasure-coded=0
        Job Counters
```

Figure: Output of agency_output part-1

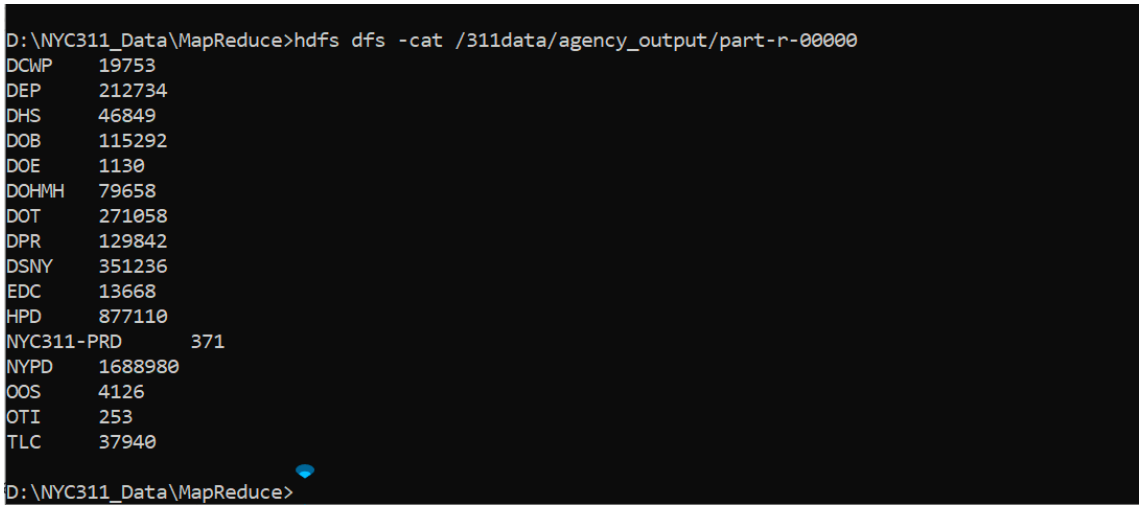


Figure: Output of agency_output part-2

3. **<u>Requests by Submission Channel:</u>** Analyzes how requests were submitted, such as online, phone, mobile or other channels.

Code:

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class ChannelCount {

   public static class ChannelMapper
         extends Mapper<Object, Text, Text, IntWritable> {

      private final static IntWritable one = new IntWritable(1);
      private Text channel = new Text();

      public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

         String line = value.toString();

         // Skip CSV header
         if (line.startsWith("\"unique_key\"")) {
```

```
            return;
        }

        // Handle commas inside quoted CSV fields
        String[] fields =
              line.split(",(?=(?:[^\"]*\"[^\"]*\")*[^\"]*$)");

        // open_data_channel_type is column 32 (index 31)
        if (fields.length > 31) {

            String channelValue = fields[31]
                  .replace("\"", "")
                  .trim();

            if (!channelValue.isEmpty()) {
               channel.set(channelValue);
               context.write(channel, one);
            }
        }
    }
}

public static class ChannelReducer
        extends Reducer<Text, IntWritable, Text, IntWritable> {

    private IntWritable result = new IntWritable();

    public void reduce(Text key,
                       Iterable<IntWritable> values,
                       Context context)
            throws IOException, InterruptedException {

        int sum = 0;

        for (IntWritable value : values) {
            sum += value.get();
        }

        result.set(sum);
        context.write(key, result);
    }
}

public static void main(String[] args)
        throws Exception {

    if (args.length != 2) {
        System.err.println(
```

```
                "Usage: ChannelCount <input path> <output path>");
            System.exit(2);
        }

        Configuration conf = new Configuration();

        Job job = Job.getInstance(
                conf,
                "NYC 311 Requests by Submission Channel");

        job.setJarByClass(ChannelCount.class);

        job.setMapperClass(ChannelMapper.class);
        job.setReducerClass(ChannelReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(
                job,
                new Path(args[0]));

        FileOutputFormat.setOutputPath(
                job,
                new Path(args[1]));

        System.exit(
                job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Execution:

```
D:\NYC311_Data\MapReduce>notepad ChannelCount.java

D:\NYC311_Data\MapReduce>javac -cp "%HADOOP_HOME%\share\hadoop\common\*;%HADOOP_HOME%\share\hadoop\common\lib\*;%HADOOP_
HOME%\share\hadoop\hdfs\*;%HADOOP_HOME%\share\hadoop\mapreduce\*;%HADOOP_HOME%\share\hadoop\yarn\*" ChannelCount.java

D:\NYC311_Data\MapReduce>jar -cvf ChannelCount.jar ChannelCount*.class
added manifest
adding: ChannelCount$ChannelMapper.class(in = 2071) (out= 920)(deflated 55%)
adding: ChannelCount$ChannelReducer.class(in = 1750) (out= 739)(deflated 57%)
adding: ChannelCount.class(in = 1666) (out= 922)(deflated 44%)

D:\NYC311_Data\MapReduce>dir ChannelCount.jar
 Volume in drive D is New Volume
 Volume Serial Number is A2F4-ACAC

 Directory of D:\NYC311_Data\MapReduce

09/12/2026  11:08 PM             3,365 ChannelCount.jar
               1 File(s)          3,365 bytes
               0 Dir(s)  140,935,380,992 bytes free

D:\NYC311_Data\MapReduce>hdfs dfs -rm -r /311data/channel_output
rm: `/311data/channel_output': No such file or directory

D:\NYC311_Data\MapReduce>hadoop jar ChannelCount.jar ChannelCount /311data/input /311data/channel_output
```

Figure: Commands for creating java file, ajr file and running jar file

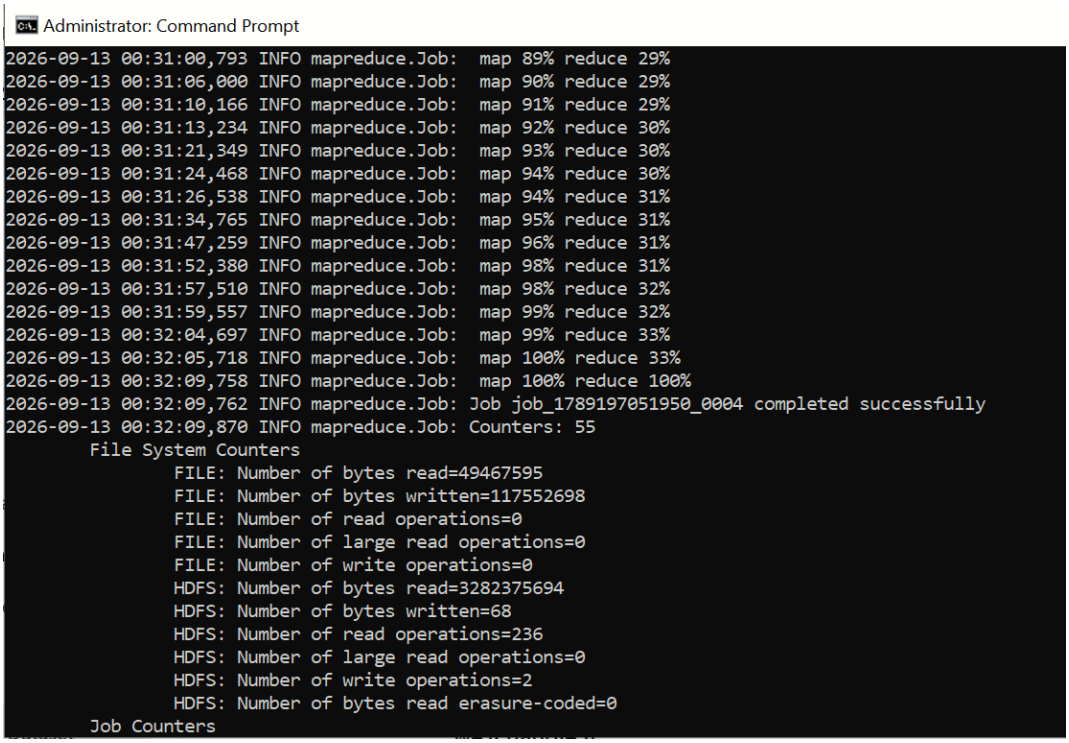


Figure: Output of channel_output part-1

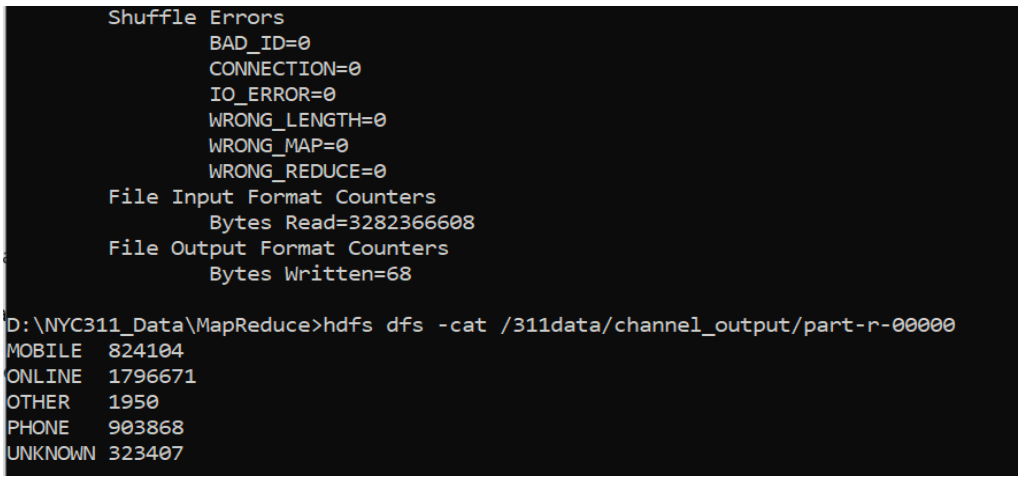


Figure: Output of channel_output part-2

Pig Analysis

1. **Top 10 Complaints:** Identifies the most frequently reported types of 311 complaints in the dataset.

Script:

top_complaints - Notepad

File Edit Format View Help

```
-- Load all NYC 311 CSV files
data = LOAD 'D:/NYC311_Data/chunks/*.csv'
       USING PigStorage(',')
       AS (
           unique_key:chararray,
           created_date:chararray,
           closed_date:chararray,
           agency:chararray,
           agency_name:chararray,
           complaint_type:chararray
       );

-- Remove repeated CSV header rows and empty complaint types
clean_data = FILTER data BY
             unique_key != 'unique_key'
             AND complaint_type IS NOT NULL
             AND complaint_type != '';

-- Group requests by complaint type
grouped_complaints = GROUP clean_data BY complaint_type;

-- Count requests for each complaint type
complaint_counts = FOREACH grouped_complaints GENERATE
                   group AS complaint_type,
                   COUNT(clean_data) AS total_requests;

-- Sort from highest to lowest
sorted_complaints = ORDER complaint_counts BY total_requests DESC;

-- Keep the top 10
top_10 = LIMIT sorted_complaints 10;

-- Display the result
DUMP top_10;
```

Execution:

```
2026-09-12 04:33:00,160 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Su
ccess!
2026-09-12 04:33:00,166 [main] WARN  org.apache.pig.data.SchemaTupleBackend - SchemaTupleBackend has already been initia
lized
2026-09-12 04:33:00,170 [main] WARN  org.apache.hadoop.io.nativeio.NativeIO - NativeIO.getStat error (3): The system can
not find the path specified.
 -- file path: tmp/temp675418607/tmp803627692/part-r-00000
2026-09-12 04:33:00,222 [main] WARN  org.apache.hadoop.io.nativeio.NativeIO - NativeIO.getStat error (3): The system can
not find the path specified.
 -- file path: tmp/temp675418607/tmp803627692/_SUCCESS
2026-09-12 04:33:00,275 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileInputFormat - Total input files to proces
s : 1
2026-09-12 04:33:00,276 [main] INFO  org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths t
o process : 1
("Illegal Parking",595463)
("Noise - Residential",428229)
("HEAT/HOT WATER",356698)
("Blocked Driveway",188778)
("Noise - Street/Sidewalk",169286)
("UNSANITARY CONDITION",130193)
("Street Condition",116290)
("PLUMBING",81430)
("Water System",72834)
("Abandoned Vehicle",68736)
2026-09-12 04:33:00,377 [main] INFO  org.apache.pig.Main - Pig script completed in 1 minute, 26 seconds and 690 millisec
onds (86690 ms)
```

Figure: Top 10 complaints types

2. **Top 10 Locations:** Determines the location types, such as streets, sidewalks, and residential buildings, where the most requests were reported.

Script:

top_locations - Notepad

File Edit Format View Help

```
-- Load NYC 311 data using proper CSV handling
data = LOAD 'D:/NYC311_Data/chunks/*.csv'
       USING org.apache.pig.piggybank.storage.CSVExcelStorage()
       AS (
           unique_key:chararray,
           created_date:chararray,
           closed_date:chararray,
           agency:chararray,
           agency_name:chararray,
           complaint_type:chararray,
           descriptor:chararray,
           descriptor_2:chararray,
           location_type:chararray
       );

-- Remove repeated headers and empty locations
clean_data = FILTER data BY
             unique_key != 'unique_key'
             AND location_type IS NOT NULL
             AND location_type != '';

-- Group requests by location type
grouped_locations = GROUP clean_data BY location_type;

-- Count requests for each location
location_counts = FOREACH grouped_locations GENERATE
                  group AS location_type,
                  COUNT(clean_data) AS total_requests;

-- Sort from highest to lowest
sorted_locations = ORDER location_counts BY total_requests DESC;

-- Display top 10 locations
top_10 = LIMIT sorted_locations 10;

DUMP top_10;
```

Execution:

```
2026-09-12 17:43:29,993 [main] WARN  org.apache.hadoop.io.nativeio.NativeIO - NativeIO.getStat error (3): The system ca
not find the path specified.
 -- file path: tmp/temp1839587430/tmp-1092292474/part-r-00000
2026-09-12 17:43:30,050 [main] WARN  org.apache.hadoop.io.nativeio.NativeIO - NativeIO.getStat error (3): The system ca
not find the path specified.
 -- file path: tmp/temp1839587430/tmp-1092292474/_SUCCESS
2026-09-12 17:43:30,101 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileInputFormat - Total input files to proces
s : 1
2026-09-12 17:43:30,101 [main] INFO  org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths t
o process : 1
(Street/Sidewalk,1152224)
(RESIDENTIAL BUILDING,877109)
(Residential Building/House,467632)
(Street,348804)
(Sidewalk,191735)
(Store/Commercial,53412)
(Park,31905)
(Club/Bar/Restaurant,24789)
(Business,18655)
(Park/Playground,16608)
2026-09-12 17:43:30,222 [main] INFO  org.apache.pig.Main - Pig script completed in 1 minute, 29 seconds and 349 millise
onds (89349 ms)

D:\NYC311_Data\PigScripts>
```

Figure: Top 10 locations

3. **Average Complaint Resolution Time:** Calculates the average time taken to resolve different types of service requests.

Script:

resolution_time - Notepad

File Edit Format View Help

```
-- Load NYC 311 data using proper CSV handling
data = LOAD 'D:/NYC311_Data/chunks/*.csv'
       USING org.apache.pig.piggybank.storage.CSVExcelStorage()
       AS (
           unique_key:chararray,
           created_date:chararray,
           closed_date:chararray,
           agency:chararray,
           agency_name:chararray,
           complaint_type:chararray
       );

-- Remove header rows and records without required dates
clean_data = FILTER data BY
             unique_key != 'unique_key'
             AND created_date IS NOT NULL
             AND created_date != ''
             AND closed_date IS NOT NULL
             AND closed_date != ''
             AND complaint_type IS NOT NULL
             AND complaint_type != '';

-- Convert date strings to date/time values
dates = FOREACH clean_data GENERATE
        complaint_type,
        ToDate(created_date, 'yyyy-MM-dd\'T\'HH:mm:ss.SSS') AS created,
        ToDate(closed_date, 'yyyy-MM-dd\'T\'HH:mm:ss.SSS') AS closed;

-- Convert dates to Unix time (seconds)
times = FOREACH dates GENERATE
        complaint_type,
        ToUnixTime(created) AS created_seconds,
        ToUnixTime(closed) AS closed_seconds;

-- Calculate resolution time in hours
resolution = FOREACH times GENERATE
             complaint_type,
             (closed_seconds - created_seconds) / 3600.0 AS resolution_hours;

-- Remove invalid negative resolution times
valid_resolution = FILTER resolution BY resolution_hours >= 0;

-- Group by complaint type
grouped = GROUP valid_resolution BY complaint_type;

-- Calculate average resolution time
average_resolution = FOREACH grouped GENERATE
                     group AS complaint_type,
                     COUNT(valid_resolution) AS total_requests,
                     AVG(valid_resolution.resolution_hours) AS average_hours;

-- Sort by average resolution time
sorted = ORDER average_resolution BY average_hours DESC;

-- Show the 10 complaint types with the longest average resolution time
top_10 = LIMIT sorted 10;

DUMP top_10;
```

Execution:

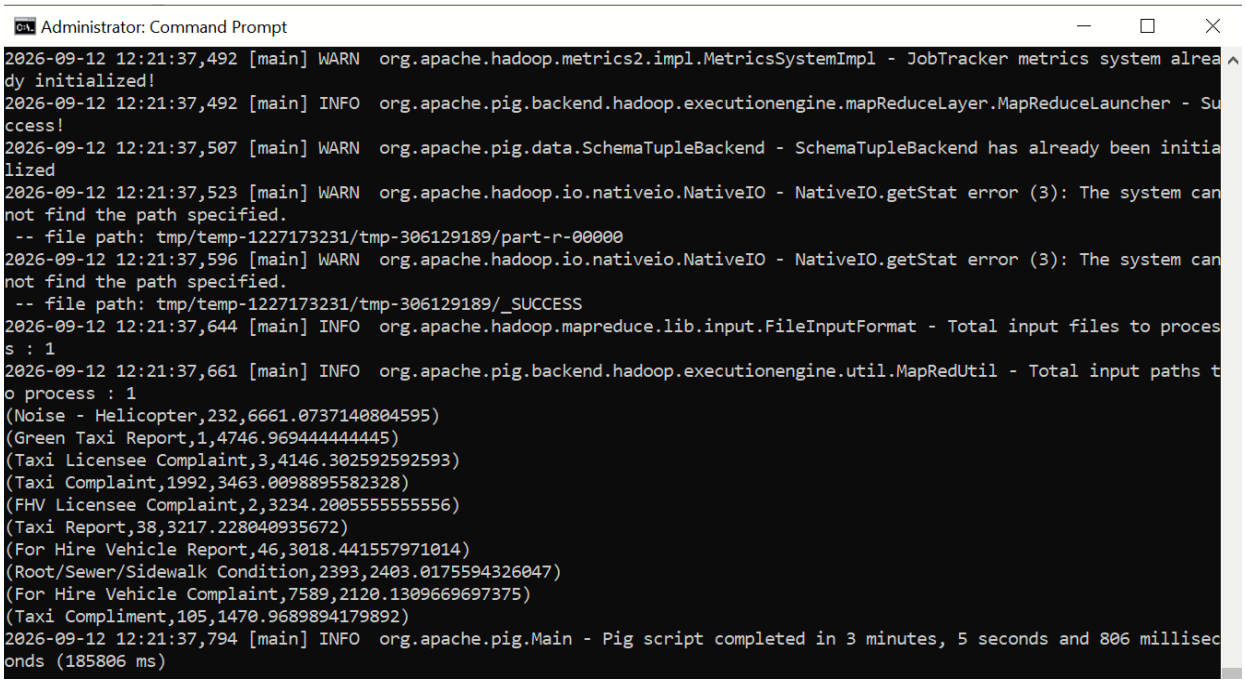


Figure: Resolution time for complaint types

4. **Monthly Trends:** Counts service requests by month to identify changes and patterns in complaint volume over time.

Script:

monthly_trends - Notepad

File Edit Format View Help

```
-- Load NYC 311 data using proper CSV handling
data = LOAD 'D:/NYC311_Data/chunks/*.csv'
       USING org.apache.pig.piggybank.storage.CSVExcelStorage()
       AS (
           unique_key:chararray,
           created_date:chararray
       );

-- Remove repeated headers and invalid dates
clean_data = FILTER data BY
             unique_key != 'unique_key'
             AND created_date IS NOT NULL
             AND created_date != '';

-- Extract Year-Month from the created date
monthly_data = FOREACH clean_data GENERATE
               SUBSTRING(created_date, 0, 7) AS year_month;

-- Group requests by month
grouped_months = GROUP monthly_data BY year_month;

-- Count requests for each month
monthly_counts = FOREACH grouped_months GENERATE
                 group AS year_month,
                 COUNT(monthly_data) AS total_requests;

-- Sort chronologically
sorted_months = ORDER monthly_counts BY year_month ASC;

-- Display monthly results
DUMP sorted_months;
```

Execution:

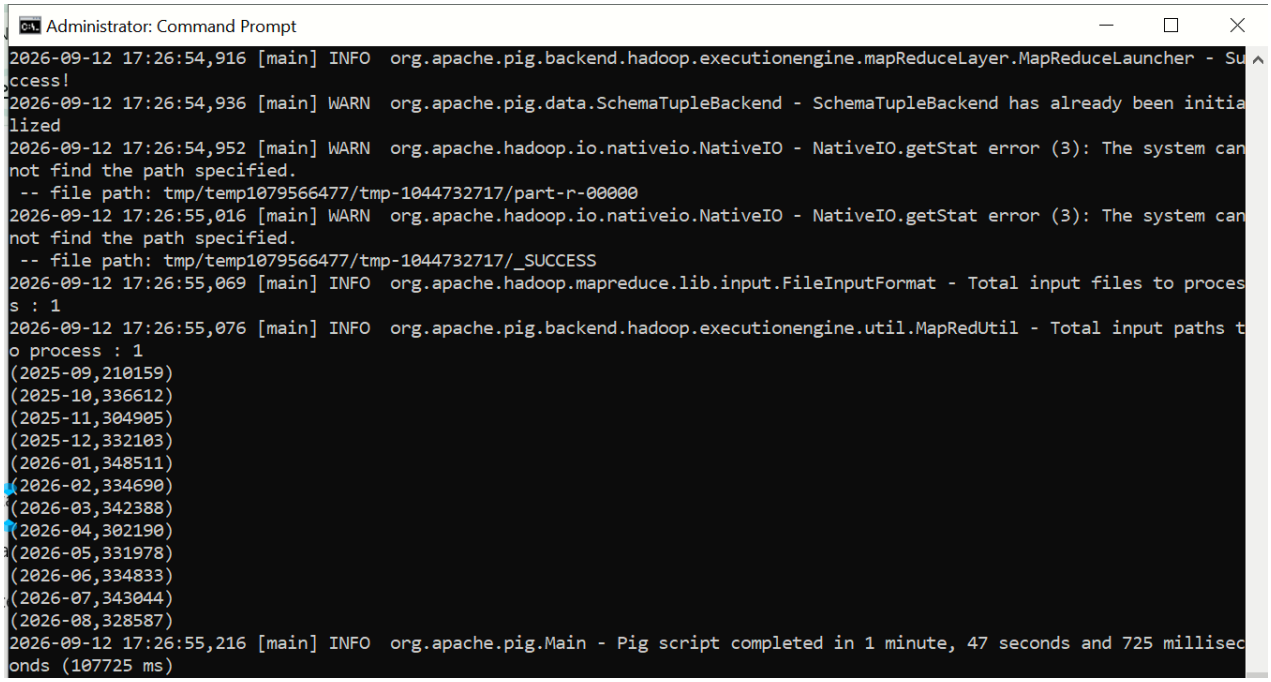


Figure: Monthly trends

5. **Complaints By Borough**: Groups and counts service requests by borough to compare the distribution of complaints across NYC.

Script:

complaints_by_borough - Notepad

File Edit Format View Help

```
        USING org.apache.pig.piggybank.storage.CSVExcelStorage()
        AS (
            unique_key:chararray,
            created_date:chararray,
            closed_date:chararray,
            agency:chararray,
            agency_name:chararray,
            complaint_type:chararray,
            descriptor:chararray,
            descriptor_2:chararray,
            location_type:chararray,
            incident_zip:chararray,
            incident_address:chararray,
            street_name:chararray,
            cross_street_1:chararray,
            cross_street_2:chararray,
            intersection_street_1:chararray,
            intersection_street_2:chararray,
            address_type:chararray,
            city:chararray,
            landmark:chararray,
            facility_type:chararray,
            status:chararray,
            due_date:chararray,
            resolution_description:chararray,
            resolution_action_updated_date:chararray,
            community_board:chararray,
            council_district:chararray,
            police_precinct:chararray,
            bbl:chararray,
            borough:chararray
        );
```

```
-- Remove repeated headers and empty borough values
clean_data = FILTER data BY
             unique_key != 'unique_key'
             AND borough IS NOT NULL
             AND borough != '';

-- Group by borough
grouped_boroughs = GROUP clean_data BY borough;

-- Count complaints
borough_counts = FOREACH grouped_boroughs GENERATE
                 group AS borough,
                 COUNT(clean_data) AS total_requests;

-- Sort highest to lowest
sorted_boroughs = ORDER borough_counts BY total_requests DESC;

DUMP sorted_boroughs;
```

Execution:

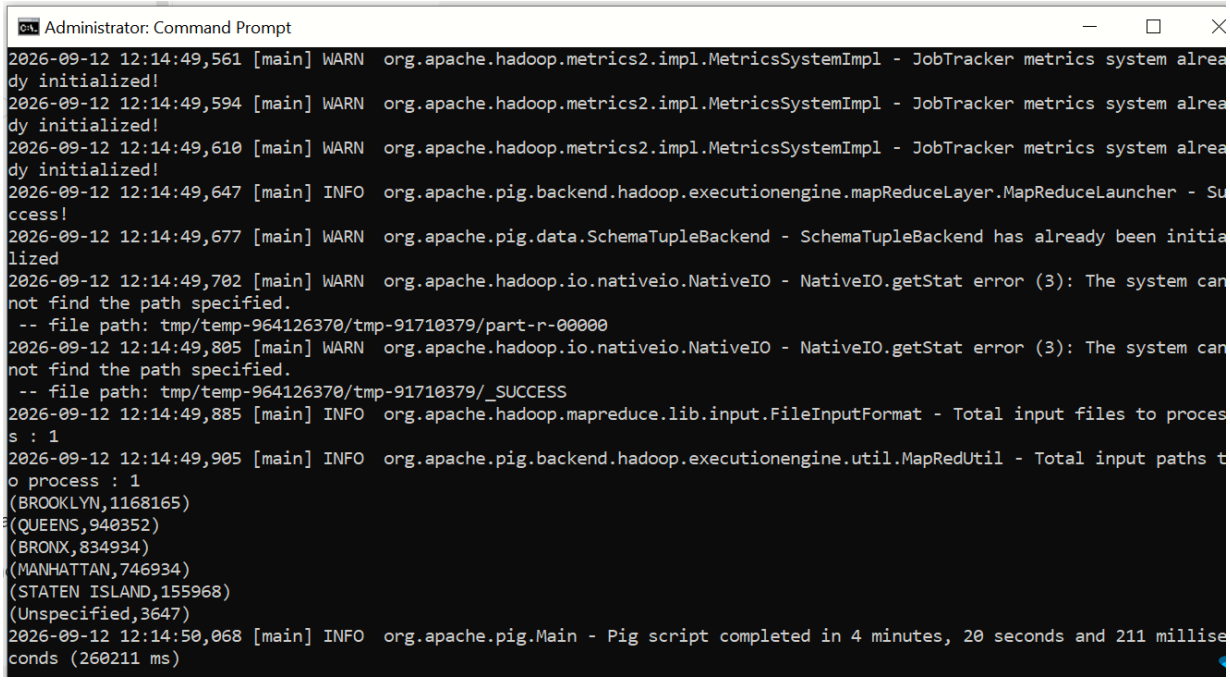


Figure: Number of complaint from borough

6. **Complaint status:** Analyzes the current status of requests, such as Closed, Open, In Progress, or Pending.

Script:

complaint_status - Notepad

File Edit Format View Help

```
data = LOAD 'D:/NYC311_Data/chunks/*.csv'
       USING org.apache.pig.piggybank.storage.CSVExcelStorage()
       AS (
            unique_key:chararray,
            created_date:chararray,
            closed_date:chararray,
            agency:chararray,
            agency_name:chararray,
            complaint_type:chararray,
            descriptor:chararray,
            descriptor_2:chararray,
            location_type:chararray,
            incident_zip:chararray,
            incident_address:chararray,
            street_name:chararray,
            cross_street_1:chararray,
            cross_street_2:chararray,
            intersection_street_1:chararray,
            intersection_street_2:chararray,
            address_type:chararray,
            city:chararray,
            landmark:chararray,
            facility_type:chararray,
            status:chararray
       );

clean_data = FILTER data BY
             unique_key != 'unique_key'
             AND status IS NOT NULL
             AND status != '';

grouped_status = GROUP clean_data BY status;

status_counts = FOREACH grouped_status GENERATE
                group AS status,
                COUNT(clean_data) AS total_requests;

sorted_status = ORDER status_counts BY total_requests DESC;

DUMP sorted_status;
```

Execution:

```
dy initialized!
2026-09-12 17:37:29,681 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Su
ccess!
2026-09-12 17:37:29,689 [main] WARN  org.apache.pig.data.SchemaTupleBackend - SchemaTupleBackend has already been initia
lized
2026-09-12 17:37:29,697 [main] WARN  org.apache.hadoop.io.nativeio.NativeIO - NativeIO.getStat error (3): The system can
not find the path specified.
 -- file path: tmp/temp803061738/tmp728297377/part-r-00000
2026-09-12 17:37:29,749 [main] WARN  org.apache.hadoop.io.nativeio.NativeIO - NativeIO.getStat error (3): The system can
not find the path specified.
 -- file path: tmp/temp803061738/tmp728297377/_SUCCESS
2026-09-12 17:37:29,793 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileInputFormat - Total input files to proces
s : 1
2026-09-12 17:37:29,796 [main] INFO  org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths t
o process : 1
(Closed,3670042)
(In Progress,101452)
(Open,58259)
(Assigned,12705)
(Pending,5774)
(Started,1606)
(Unspecified,162)
2026-09-12 17:37:29,959 [main] INFO  org.apache.pig.Main - Pig script completed in 2 minutes, 8 seconds and 247 millisec
onds (128247 ms)
```

Figure: Complaint status

Conclusion

This project showed how Big Data technologies can be used to process and analyze a large real world dataset such as the NYC 311 Service Requests dataset. HDFS was used to store approximately 3.85 million service request records in a distributed Hadoop environment. The results provided useful insights into the nature and distribution of public service requests. For example, Illegal Parking was the most common complaint type, while Brooklyn had the highest number of requests among the boroughs. The analysis also showed that ONLINE was the most frequently used submission channel and that most requests were marked as Closed.